

Autodesk® Scaleform®

HUD

AAA

C++

Scaleform 4.2 HUD
Flash UI

Nate Mitchell Prasad Silva

2.0

2010 7 30

Copyright Notice

Autodesk® Scaleform® 4.2

© 2012 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and other countries: 123D, 3ds Max, Algor, Alias, AliasStudio, ATC, AUGI, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk Homestyler, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSketch, AutoSnap, AutoTrack, Backburner, Backdraft, Beast, Beast (design/logo) Built with ObjectARX (design/logo), Burn, Buzzsaw, CAiCE, CFdesign, Civil 3D, Cleaner, Cleaner Central, ClearScale, Colour Warper, Combustion, Communication Specification, Constructware, Content Explorer, Creative Bridge, Dancing Baby (image), DesignCenter, Design Doctor, Designer's Toolkit, DesignKids, DesignProf, DesignServer, DesignStudio, Design Web Format, Discreet, DWF, DWG, DWG (design/logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DWFx, DXF, Ecotect, Evolver, Exposure, Extending the Design Team, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, Freewheel, GDX Driver, Green Building Studio, Heads-up Design, Heidi, Homestyler, HumanIK, i-drop, ImageModeler, iMOUT, Incinerator, Inferno, Instructables, Instructables (stylized robot design/logo), Inventor, Inventor LT, Kynapse, Kynogon, LandXplorer, Lustre, MatchMover, Maya, Mechanical Desktop, MIMI, Moldflow, Moldflow Plastics Advisers, Moldflow Plastics Insight, Moondust, MotionBuilder, Movimento, MPA, MPA (design/logo), MPI (design/logo), MPX, MPX (design/logo), Mudbox, Multi-Master Editing, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, Pixlr, Pixlr-o-matic, PolarSnap, Powered with Autodesk Technology, Productstream, ProMaterials, RasterDWG, RealDWG, Real-time Roto, Recognize, Render Queue, Retimer, Reveal, Revit, RiverCAD, Robot, Scaleform, Scaleform GfX, Showcase, Show Me, ShowMotion, SketchBook, Smoke, Softimage, Sparks, SteeringWheels, Stitcher, Stone, StormNET, Tinkerbox, ToolClip, Topobase, Toxik, TrustedDWG, T-Splines, U-Vis, ViewCube, Visual, Visual LISP, Vtour, WaterNetworks, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. "AS IS." AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF

MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

Autodesk Scaleform

	HUD
	Autodesk Scaleform Corporation
	6305 Ivy Lane, Suite 310
	Greenbelt, MD 20770, USA
	www.scaleform.com
Email	info@scaleform.com
	+1 (301) 446-3200
Fax	+1 (301) 446-3199

5.1.1	Minimap	34
6	性能解析	36
6.1		36
6.2		38

1 はじめに

Scaleform HUD

Direct Access API

AAA

Scaleform®
UI

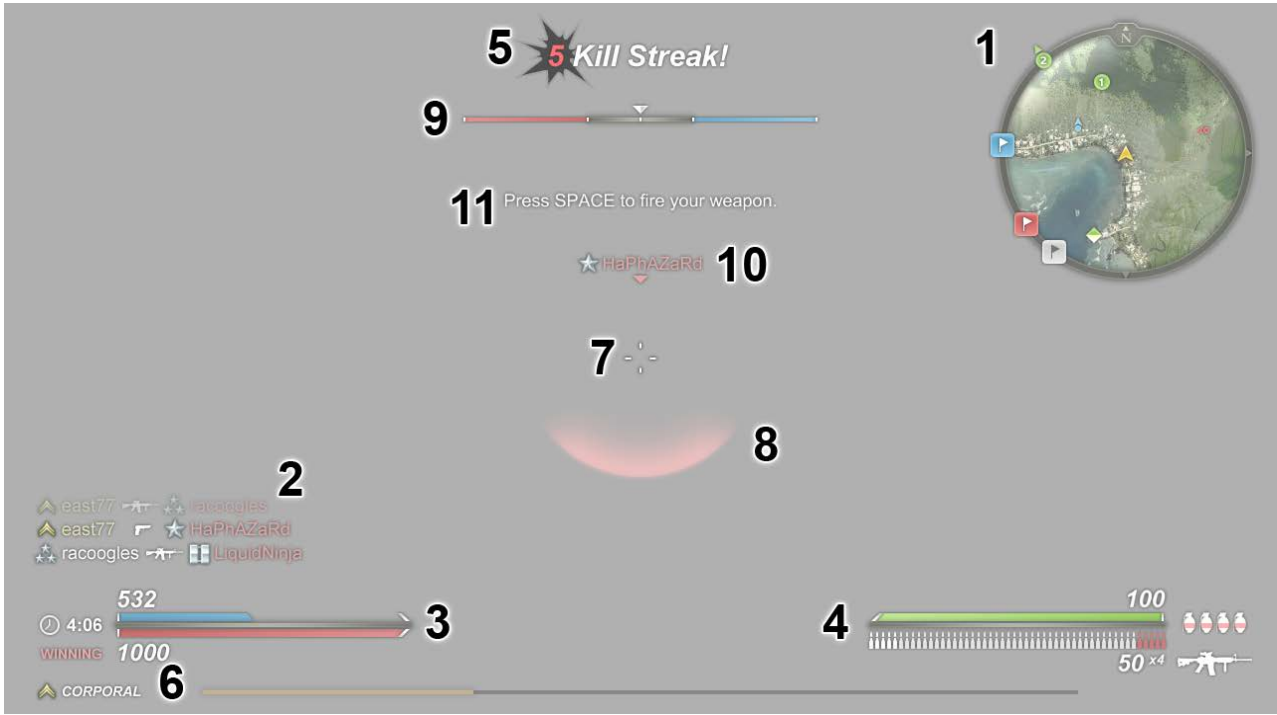


図 1: HUD の概観

HUD

UI

1. minimap

2.

3.

4.

5.

6.

7.

8.

9.

10.

11.

FPS

UI

Scaleform

2 概要

2.1 ファイルの位置とビルドの注記

- *Apps\Kits\HUD* - HUD C++
- *Bin\Data\AS2 or AS3\Kits\HUD* - Flash ActionScript
(AS2 ActionScript2/Flash8 AS3
ActionScript3/Flash10)
- *Projects\Win32 \{Msvc80 または Msvc90}\Kits\HUD* – Microsoft Windows
Visual Studio 2005/2008
- *Projects\Xbox360\{Msvc80 または Msvc90}\Kits\HUD* – Xbox 360 Visual
Studio 2005/2008
- *Projects\Common\HudKit.mk* – HUD PS3 makefile

Windows HUDKit.exe *Bin\Kits\HUD*
Scaleform SDK

Windows *Projects\Win32\Msvc80\Kits*
Msvc90\Kits Scaleform 4.2 Kits.sln

Bin\Data\AS2\Kits\HUD or Bin\Data\AS3\Kits\HUD

Xbox 360 *Projects\Xbox360\{Msvc80*
Msvc90}\Kits\HUD Scaleform 4.2 Kits.sln
Bin\Data\AS2 or AS3\Kits\HUD
Xbox360

PS3 make Scaleform
HUD

PS3 SN
Bin\PS3

o app_home/ Directory: *{Local Scaleform Directory}\Bin\Data\AS2 or AS3\Kits\HUD*

2.2 デモの使い方

HUD

2

minimap

1

500

3

1

15

×1

×1

×4

2.3 制御方法

2.3.1 キーボード (Windows)

W、A、S、D	W S A D
スペース	
R	
G	
ESC	
B	AI
1、2	• 1 • 2

2.3.2 コントローラ (Xbox/PS3)

Dパッド 左ジョイスティック	W S A D
右トリガー	
X / □	
左トリガー	
B / ○	
Y / △	

3 アーキテクチャ

HUD Flash UI HUD
HUD C++ C++

3.1 C++

HUD MVC HUD C++ Flash UI
Fx

HUD HUD HUD HUD

HUD

Scaleform Direct Access API
Scaleform

UI

C++ *Apps\Kits\HUD*

3.1.1 デモ

- **HUDKitDemo.cpp** – Scaleform Player C++ SWF
- **HUDKitDemoConfig.** – FxPlayerConfig.h HUDKitDemo
Windows

3.1.2 HUD/Minimap ビューコア

- **FxHUDKit.h** – HUD HUD
- **FxHUDView.h/.cpp** – HUD
- **FxMinimap.h** – minimap
- **FxMinimapView.h/.cpp** – minimap

3.1.3 ゲームシミュレーション

- **HUDAdapter.h/.cpp** – HUD
- **HUDEntityController.h/.cpp** – AI
- **HUDSimulation.h/.cpp** – UI

3.2 Flash

HUD Flash
 FLA **HUDKit.fla** **Minimap.fla** 2
 Scaleform HUD UI
 FLA UI

Scaleform Flash C++ ActionScript Flash
 HUD Classic Tweens
 C++ C++
 GfX::Value::GotoAndPlay()

3.2.1 HUDKit.fla

Bin\Data\AS2 or AS3\Kits\HUD HUDKit.fla
 SWF HUDKitDemo
 HUDKit.fla minimap Flash UI

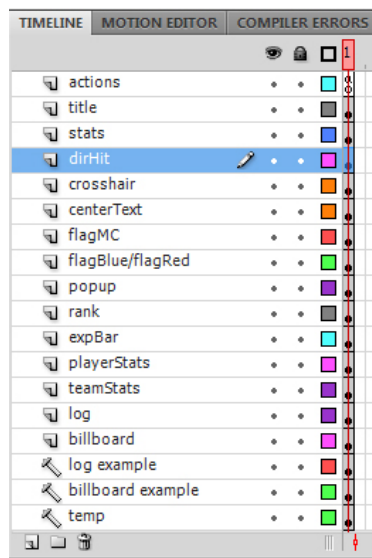


図 2: HUDKit.fla のレイヤー

HUDKitDemo

Frame 1

External Interface

HUDKit MovieClip

```
ExternalInterface.call("registerHUDView", this);
```

MovieClip

C++ C++ HUDView

MovieClip

HUD View

Scaleform C++

3.2.2 Minimap.fla

図 3: Minimap のシンボル

Minimap.fla Minimap

Minimap
3D

minimap
minimap 5

-
-
-
-

C++ Stage Direct Access API MovieClip Direct Access API

Minimap

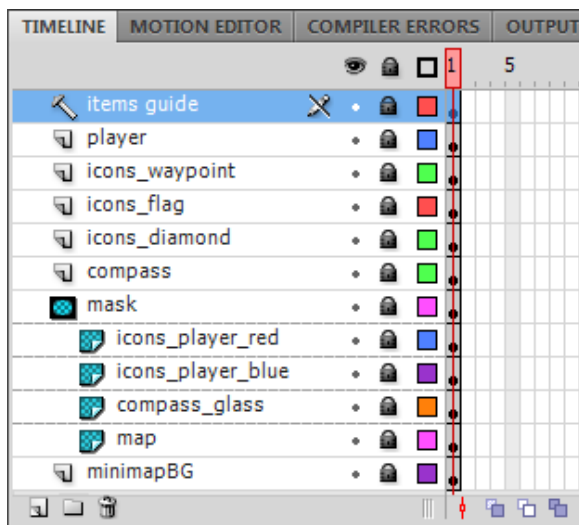


図 4: 「minimap」シンボルのレイヤー構造

- **items guide:**
SWF
 - **player:**
 - **icons_waypoint:**
 - **icons_flag:**
 - **icons_diamond:**
 - **compass:**
 - **mask:**
 - icons_player_red:
 - icons_player_blue:
 - compass_glass:
 - map:
- C++

- **minimapBG:**

C++

Minimap	com.scaleform.Minimap	<i>Bin\Data\AS2 or</i>
<i>AS3\Kits\HUD \com\scaleform</i>		
HUDKitDemo	Minimap	
minimap		

4 HUD ビュー

C++ HUD View

HUDKit.swf

- FxHUDView – HUD HUDKit.swf MovieClip
- BillboardCache – minimap
- FxHUDLog – HUD FxHUDMessages
- FxHUDMessage – MovieClip

FxHUDView

MovieClip

MovieClip

MC

FxHUDView

UpdateView()
MovieClip

MovieClip

HUDKit.fla

UI

FxHUDView::UpdateView()

UpdateTeamStats() UpdatePlayerEXP() Update
PlayerStats() UpdateEvents() UpdateTutorial() UpdateBillboards()
UI

MovieClip
MovieClip

UpdateView()

MovieClip

Scaleform Direct Access API

Flash

GFx::Value

HUD View

Flash

GFx::Value::SetDisplayInfo()

GFx::Value::GetDisplayInfo()

GFx::Value::SetText()

MovieClip

4.1 ラウンドとの統計とスコアボード

HUDKit.fla	Scene 1	teamStats		teamStats MovieClip
			MovieClip	textField redWinning
blueWinning	2	textField		scoreRed scoreBlue
	teamRed	teamBlue		textField roundTime

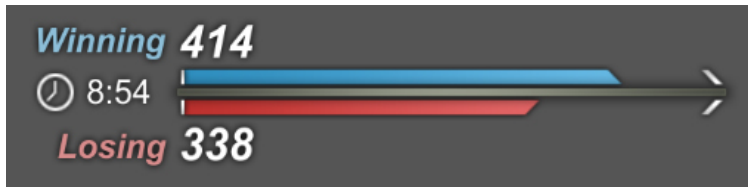


図 5: teamStats シンボル

teamStats	FxHUDView::UpdateTeamStats()	teamStats
MovieClip		

```
void FxHUDView::UpdateTeamStats(FxHUDEnvironment *penv)
{
    // Retrieve the scores from the simulation.
    unsigned scoreBlue = unsigned(penv->GetHUDBlueTeamScore());
    unsigned scoreRed = unsigned(penv->GetHUDRedTeamScore());

    String text;
    Value tf;
    Value::DisplayInfo info;

    // We won't update any element who has not changed since the last update
    // to avoid unnecessary updates. To do so, we compare the previous
    // simulation State to the latest simulation data. If the two are
    // different, update the UI element in the HUD View.
    if (State.ScoreRed != scoreRed)
    {
        Format(text, "{0}", scoreRed);
        ScoreRedMC.SetText(text); // Update the red team score text.

        info.SetScale((scoreRed / (Double)MaxScore) * 100, 100);
        TeamRedMC.SetDisplayInfo(info); // Update and scale the red team
                                        // score bar movieClip.
    }

    if (State.ScoreBlue != scoreBlue)
    {
        Format(text, "{0}", scoreBlue);
        ScoreBlueMC.SetText(text); // Update the blue team score text.

        info.SetScale((scoreBlue / (Double)MaxScore) * 100, 100);
        TeamBlueMC.SetDisplayInfo(info); // Update and scale the blue team
```

```

        score bar movieClip.
    }

    if (State.ScoreRed != scoreRed || State.ScoreBlue != scoreBlue)
    {
        // Update the "Winning" and "Losing" text fields for both teams
        // if the score has changed.
        if (scoreBlue > scoreRed)
        {
            RedWinningMC.SetText(LosingText);
            BlueWinningMC.SetText(WinningText);
        }
        else
        {
            RedWinningMC.SetText(WinningText);
            BlueWinningMC.SetText(LosingText);
        }
    }

    // Update the roundtime clock
    float secondsLeft = penv->GetSecondsLeftInRound();
    unsigned sec = ((unsigned)secondsLeft % 60);
    if (sec != State.Sec)
    {
        unsigned min = ((unsigned)secondsLeft / 60);
        String timeLeft;
        Format(timeLeft, "{0}:{1:0.2}", min, sec);
        RoundTimeMC.SetText(timeLeft);
        State.Sec = sec;
    }
}

```

Scaleform::String		GfX::Value::SetText()	scoreRed
scoreBlue	textField	teamRed	teamBlue
GfX::Value::SetDisplayInfo()		X	0
		100	
blueWinning/redWinning	textField		2
	GfX::Value::SetText()	HUDView	
Winning	Losing		
textField		UpdateView()	
		1	GfX::Value::SetText()
roundTime	textField	String	

```
HUDKit.fla    Scene 1    playerStats    playerStats
textField    healthN    health    weapon    HUD
reload
```



1	51	Keyframe GfX::Value::GotoAndStop()	Keyframe MovieClip
---	----	---------------------------------------	-----------------------

- | | | | | | |
|---------------------------|---|-----------|----------|---|----------|
| Frame 1 | 4 | | | 5 | Keyframe |
| GfX::Value::GotoAndStop() | | MovieClip | Frame 5 | | |
| | | | Keyframe | | |

- playerStats*

15

```

unsigned ammoInClip = pplayer->GetNumAmmoInClip();
unsigned ammoClips = pplayer->GetNumClips();
unsigned clipSize = pplayer->GetMaxAmmoInClip();
unsigned grenades = pplayer->GetNumGrenades();

unsigned ammoFrames = 51;
unsigned grenadeFrames = 5;

String text;

if(pplayer->GetHealth() != State.Health)
{
    unsigned health = unsigned(pplayer->GetHealth() * 100);
    Format(text, "{0}", health);
    HealthNMC.SetText(text); // Update the health text field.

    Value::DisplayInfo info;
    info.SetScale(Double(health), 100);
    HealthMC.SetDisplayInfo(info); // Update and scale the health bar.
}

if(ammoInClip != State.AmmoInClip)
{
    Format(text, "{0}", ammoInClip);
    AmmoNMC.SetText(text); // Update the ammo text field.

    // The ammo is divided into two parts: the white bullet icons (AmmoMC)
    // and their respective backgrounds (AmmoBGMC).

    // To update the number of bullets currently in the clip, we use
    // GotoAndStop with AmmoMC and select the frame with the proper
    // number of white bullet icons. There are 51 frames in AmmoMC
    // (defined above in ammoFrames) and the first frame displays every
    // bullet icon. To display X number of bullets, we subtract X from
    // the total number of frames in AmmoMC (51) and GotoAndStop on the
    // result.
    AmmoMC.GotoAndStop(ammoFrames - ammoInClip);
}

if(ammoClips != State.AmmoClips)
{
    Format(text, "x{0}", ammoClips);
    ClipNMC.SetText(text); // Update the remaining ammo clips text field.
}

if(grenades != State.Grenades)
{

```

```

        // Grenades are setup similar to Ammo. The first frame for the
        // Grenades movieClip shows 4 white grenade icons. The second frame
        // shows 3.

        // To display X number of grenades, we subtract X from the total
        // number of frames in GrenadeMC (5) and GotoAndStop on the result.
        // Note: GrenadeMC actually contains 10 frames but 6-10 are unused by
        // this implementation.
        GrenadeMC.GotoAndStop(grenadeFrames - grenades);
    }

    if (pplayer->GetWeaponType() != State.weaponType)
    {
        // Change the weapon icon if the player has changed weapons.
        unsigned weaponFrame = GetWeaponFrame(pplayer->GetWeaponType());
        WeaponMC.GotoAndStop(weaponFrame);

        // Update the ammo background icons based on clipSize.
        if (clipSize != State.AmmoClipSize)
            AmmoBGMC.GotoAndStop(ammoFrames - clipSize);
    }

    // If the ammo in clip is less than 6 bullets, play the "Reload" indicator
    // movieClip.
    if (ammoInClip <= 5)
    {
        // ReloadMC will play until we GotoAndStop on Frame 1, so no need to
        // start the animation more than once.
        if (!bReloadMCVisible)
        {
            bReloadMCVisible = true;
            ReloadMC.GotoAndPlay("on"); // Will cycle back to "on" frame
                                         when it reaches the end and play.
                                         again
        }
    }

    // If the reload indicator is displayed but there are more than six bullets
    // hide it and stop the animation by playing frame 1 of the movieClip.
    else if (bReloadMCVisible)
    {
        ReloadMC.GotoAndStop("1"); // Frame 1 contains stop();
        bReloadMCVisible = false;
    }
}

```

<i>ammo</i>	Frame 1	50		Frame 51	0
			Keyframe		
	ammoMC.GotoAndStop(ammoFrames - ammoInClip)				HUD
	1				
<i>ammoBG</i>	ammoBGMC.GotoAndStop()			X	
	Keyframe			X	
		50		ammoBG.GotoAndStop(1)	50
	Frame 1			MovieClip	
	FxHUDView::UpdateView()				

4.3 フラグキャプチャインジケータ

HUDKit.fla Scene 1 flag *flagMC*

CaptureState *arrow* 2

flag

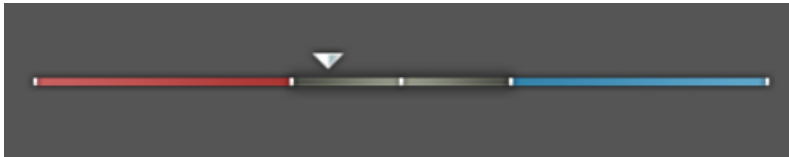


図 7: *flagMC* シンボル

flagMC *flagMC* 5 Classic Tween Flash

Hidden Visible Fade-In FadeOut 4 Keyframe

GotoAndStop() Keyframe labels/numbers 1 5 "On" "Off"

FxHUDView::UpdateEvents() *flagMC*

arrow

```
// Flag capture indicators and reticule behavior.
void FxHUDView::UpdateEvents(FxHUDEnvironment *penv)
{
    Value::DisplayInfo info;

    if (penv->IsHUDPlayerCapturingObjective())
    {
        if(!bFlagMCVisible)
        {
            SetVisible(&FlagMC, true); // Set the Flag MovieClip's
                                         visibility to true.
            FlagMC.GotoAndPlay("on"); // Play the fade-in animation once.
            bFlagMCVisible = true;
        }

        // Shift the x-coordinate of the Flag Arrow to show the capture state
        // of the flag the player is currently capturing.
        info.SetX(penv->GetHUDCaptureObjectiveState() * 177);
        FlagArrowMC.SetDisplayInfo(info);
        info.Clear();
    }
}
```



```

    }
    else if (bFlagMCVisible & !penv->IsHUDPlayerCapturingObjective())
    {
        // If the flag indicator is still visible and the player is
        // no longer capturing an objective, play the fade-out animation
        FlagMC.GotoAndPlay("off");
        bFlagMCVisible = false;
    }
}

```

flagMC.GotoAndPlay("on")	flagMC		flagMC
Visible	flagMC.GotoAndPlay("off")		flagMC
	Frame 11 20		Frame 20
Frame 1		Flash	
	ActionScript C++	Frame 1	stop();
	MovieClip Frame 1	Frame 1	Alpha 0

arrow [GfX::Value::SetDisplayInfo\(\)](#)
 CaptureState X GfX::Value::DisplayInfo

HUDKit.fla Scene 1 crosshair
 reticule

```
reticule
left  right  bottom  top    4
```

PlayerFire



図 8: レティクルシンボル

	FxHUDView::UpdateEvents()	FxHUDView::OnEvent()
2	ReticuleHeat	PlayerFire
FxHUDEvent	ReticuleFiringOffset	

```
// If the player is firing his/her weapon, update the reticule's heat.
// The reticule elements are updated in FxHUDView::UpdateEvents() based on
// the ReticleHeat and the ReticleFiringOffset.
```

```
case FxHUDEvent::EVT_PlayerFire:
{
    if (ReticleHeat < 8)
        ReticleHeat += 2.5f;

    ReticleFiringOffset = 2;
}
```

[illegible]

```
// Shift the XY-coordinates of the reticule elements based on the
// firing duration and rate of fire.
```

```
if (ReticleHeat > 0 || ReticleFiringOffset > 0){
    if (ReticleHeat > 1.0f)
```

```

        ReticleHeat -= .02f;

ReticleFiringOffset -= .02f;
if (ReticleFiringOffset < 0)
    ReticleFiringOffset = 0;

info.SetY((-6) - (ReticleFiringOffset + ReticleHeat));
ReticuleMC_Top.SetDisplayInfo(info);
info.Clear();

info.SetX((6) + (ReticleFiringOffset + ReticleHeat));
ReticuleMC_Right.SetDisplayInfo(info);
info.Clear();

info.SetX((-6) - (ReticleFiringOffset + ReticleHeat));
ReticuleMC_Left.SetDisplayInfo(info);
info.Clear();

info.SetY((6) + (ReticleFiringOffset + ReticleHeat));
ReticuleMC_Bottom.SetDisplayInfo(info);
info.Clear();
}

```

4.5 方向性ヒットインジケータ



- a)
- b)

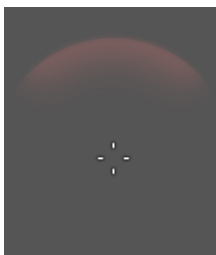
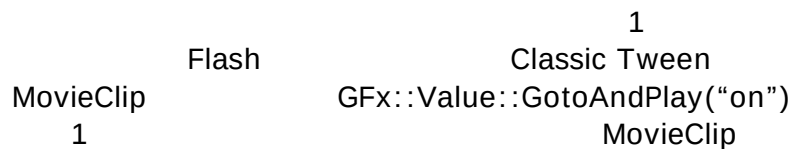


図 9: dirHit シンボル



```

void FxHUDView::OnEvent( FxHUDEvent* pevent )
{
    switch ( pevent->GetType() )
    {
        // If the Damage Event (Player was hit) is fired, show the
        // appropriate directional hit indicator.
        // We can use GotoAndPlay("on") to play the fade-in animation. It
        // will fade out on its own after ~1 second (this animation is setup
        // on the Flash timeline)
        case FxHUDEvent::EVT_Damage:
        {
            FxHUDDamageEvent* peventDamage = (FxHUDDamageEvent*)pevent;
            float dir = peventDamage->GetDirection();
            if (dir > 0)
            {
                if (dir > 90)
                {

```

```

        if (dir > 135)
            DirMC_B.GotoAndPlay("on");
        else
            DirMC_BR.GotoAndPlay("on");
    }
    else
    {
        if (dir > 45)
            DirMC_TR.GotoAndPlay("on");
        else
            DirMC_T.GotoAndPlay("on");
    }
}
else
{
    if (dir < -90)
    {
        if (dir < -135)
            DirMC_B.GotoAndPlay("on");
        else
            DirMC_BL.GotoAndPlay("on");
    }
    else
    {
        if (dir < -45)
            DirMC_L.GotoAndPlay("on");
        else
            DirMC_TL.GotoAndPlay("on");
    }
}
break;
}
}

```

4.6 ランクと経験値の表示

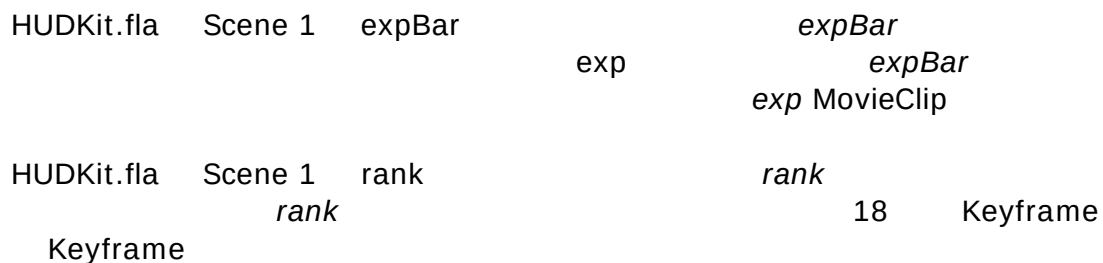
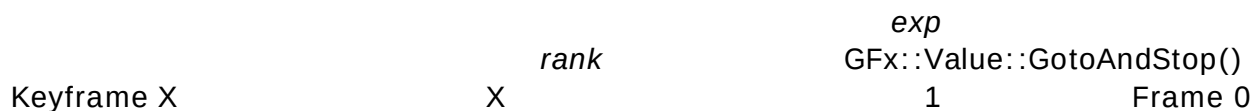


図 10: expBar と rank シンボル

```

void FxHUDView::UpdatePlayerEXP(FxHUDEnvironment *penv)
{
    FxHUDPlayer* pplayer = penv->GetHUDPlayer();
    if (pplayer->GetLevelXP() != State.Exp)
    {
        Value::DisplayInfo info;
        // Update the rank icon. Each rank icon is on a different frame of
        // the rank movieClip.
        RankMC.GotoAndStop(UInt(pplayer->GetRank() + 1));
        info.SetScale(pplayer->GetLevelXP() * 100, 100); // Scale the EXP
                                                         bar movieClip.
        ExpBarMC.SetDisplayInfo(info);
    }
}
  
```



4.7 テキスト通知

HUDKit.fla Scene 1 centerText

centerTextMC
textField *centerText*

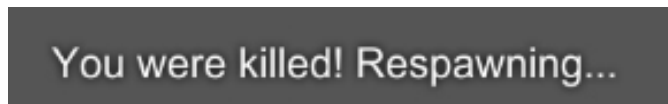
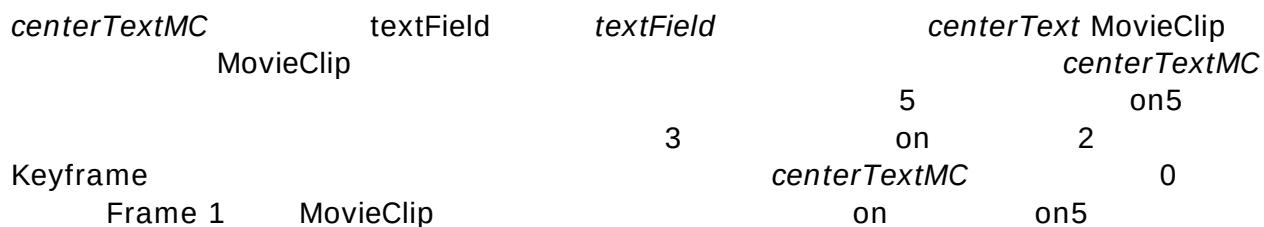


図 11: centerTextMC シンボル



```
TextFieldMC.SetText(RespawnText); // Set the text field of the Center Text.
CenterTextMC.GotoAndPlay("on"); // Play the fade-in animation, it will fade on
its own after ~3 sec.
```

centerTextMC *textField* MovieClip GotoAndPlay("on")
MovieClip Classic Tween 3

4.8 ポップアップと通知

HUDKit.fla Scene 1 popup popup 3

C++ 3 5



図 12: popup シンボル

Popup
Classic Tweens popup
popupNumber popupText popupBG 4

popup actions
popupBG
popupText popupNumber

textField

on Keyframe GotoAndPlay()

0 Frame 1

```
// If a KillStreak event is fired, display the kill streak pop-up.
case FxHUDEvent::EVT_KillStreak:
{
    FxHUDKillStreakEvent* peventKS = (FxHUDKillStreakEvent*)pevent;

    // Format the number of kills
    String text;
    Format(text, "{0}", peventKS->GetSrcKillStreak());
    PTextFieldMC.SetText(text); // Set the text field of the PopUp.

    // Play the fade-in animation, it will fade on its own after ~4 sec.
    PopupMC.GotoAndPlay("on");
}
break;
```

KillStreak FxHUDEvent

textField PTextFieldMC C++ GotoAndPlay("on") popupText

popup

4.9 メッセージとイベントログ

HUDKit.fla Scene 1 log

log

HUD

3



図 13: log と logMessage シンボル

logMessage

3

logMessage

C++

htmlText

textField

FxHUDMessage::Movieclip* FxHUDLog::GetUnusedMessageMovieclip()

LogMessage MovieClip

MovieClip

Log MovieClip

FxHUDMessage::Movieclip* FxHUDLog::GetUnusedMessageMovieclip()

{

if (MessageMCs.IsEmpty())

{

// Request a new line in the SWF

Value logline;

LogMC.AttachMovie(&logline, "LogMessage", "logMessage"+NumMessages);

FxHUDMessage::Movieclip* pmc = new FxHUDMessage::Movieclip(logline);

MessageMCs.PushBack(pmc); // Add new Message MC to list of unused
message movieClips.

}

FxHUDMessage::Movieclip* pmc = MessageMCs.GetFirst(); // Use an unused
Message movieClip.

pmc->SetVisible(true);

MessageMCs.Remove(pmc);

return pmc;

```
}
```

```
                                C++
                                You are now a [Rank Icon] [Rank Name]
                                Level Up
htmlText      C++
```

```
void FxHUDLog::Process(FxHUDEvent *pevent)
{
    String msg;
    switch (pevent->GetType())
    {
        case FxHUDEvent::EVT_LevelUp:
        {
            FxHUDLevelUpEvent* e = (FxHUDLevelUpEvent*)pevent;
            Format(msg, "You are now a <img src='rank{1}'/> {0}!",
                e->GetNewRankName(), e->GetNewRank());
        }
        break;
    }
}

FxHUDMessage::Movieclip* pmc = GetUnusedMessageMovieclip();
FxHUDMessage* m = new FxHUDMessage(msg, pmc); // 3 seconds
Log.PushBack(m);
NumMessages++;
```

HUD Update

```
void FxHUDLog::Update()
{
    SF_ASSERT(LogMC.IsDisplayObject());

    Double rowHeight = 30.f;

    // Tick the message lifetimes
    Double yoff = 0;
    FxHUDMessage* data = Log.GetFirst();
    while (!Log.IsNull(data))
    {
        FxHUDMessage* next = Log.GetNext(data);
        if (data->IsAlive())
        {
            // Layout mgmt and animation
            Value::DisplayInfo info;
```

```

        info.SetY(yoff);
        data->GetMovieclip()->GetRef().SetDisplayInfo(info);
    }
    data = next;
    yoff += rowHeight;
}

// Layout management and animation
Value::DisplayInfo info;
info.SetY(LogOriginalY - (NumMessages * rowHeight));
LogMC.SetDisplayInfo(info);

// Remove dead messages
data = Log.GetFirst();
while (!Log.IsNull(data))
{
    FxHUDMessage* next = Log.GetNext(data);
    if (!data->IsAlive())
    {
        Log.Remove(data);
        NumMessages--;

        FxHUDMessage::Movieclip* pmc = data->GetMovieclip();
        pmc->SetVisible(false);
        MessageMCs.PushBack(pmc);

        delete data;
    }
    data = next;
}
}

```

Log	Alpha	0	
FxHUDMessage			Log
	FxHUDMessage	MovieClip	MessageMC
FxHUDMessage		GFx::Value::DisplayInfo	.SetY()
GFx::Value::SetDisplayInfo()			MessageMC

4.10 ビルボード

HUDKit.fla Scene 1 billboard

billboard_container

3D

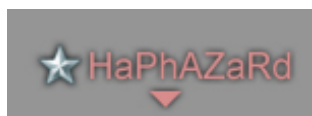


図 14: billboard_enemy シンボル

			GfX::Value::SetDisplayInfo()			
			2D	MovieClip	UI	
	MovieClip	billboard_friendly	billboard_enemy	text		
	billboard_label_	friendly/enemy	arrow			
		billboard_label_*			textField	
	billboard_friendly	Show	Keyframe	billboard_friendly		Classic
Tween					Show	

MovieClip C++
 FxHUDView::BillboardCache::GetUnusedBillboardMovieclip

```

if (UnusedBillboards.IsEmpty())
{
    Value::DisplayInfo info(false);

    Value billboard, temp;
    String instanceName;
    Format(instanceName, "{0}{1}", BillboardPrefix, BillboardCount++);
    CanvasMC.AttachMovie(&billboard, SymbolLinkage, instanceName);

    BillboardMC* pbb = new BillboardMC();
    pbb->Billboard = billboard;
    billboard.GetMember("label", &temp);
    temp.GetMember("textField", &temp);
    pbb->Textfield = temp;

    pbb->Billboard.SetDisplayInfo(info);

```

```

        UnusedBillboards.PushBack(pbb);
    }

```

```

BillboardCache      CanvasMC      MovieClip
                    billboard_enemy/billboard_friendly      BillboardCache
CanvasMC.AttachMovie()      MovieClip      MovieClip

```

```

Billboard      UpdateBillboards()      BeginProcessing()      EndProcessing()
GetUnusedBillboardMovieclip()      BeginProcessing()
EndProcessing()

```

```

    UpdateBillboards()

```

```

// Process friendlies
// Friendly billboards will always show names

```

```

BillboardMC* pbb = NULL;

```

```

FriendlyBillboards.BeginProcessing();
for (unsigned i=0; i < friendlies.GetSize(); i++)
{
    info.Clear();
    if (FriendlyBillboards.GetUnusedBillboardMovieclip(friendlies[i].pPlayer,
                                                         &pbb))
    {
        info.SetVisible(true);
        pbb->Billboard.GotoAndPlay("show");
        // Load player info
        String title;
        Format(title,
               "<img src='rank{0}' width='20' height='20' align='baseline'
               vspace='-7' /> {1}",
               friendlies[i].pPlayer->GetRank()+1,
               friendlies[i].pPlayer->GetName());
        pbb->Textfield.SetTextHTML(title);
    }
    info.SetX(friendlies[i].X);
    info.SetY(friendlies[i].Y);
    info.SetScale(friendlies[i].Scale, friendlies[i].Scale);
    pbb->Billboard.SetDisplayInfo(info);
}

```

```
FriendlyBillboards.EndProcessing();
```

```
UpdateBillboards()
```

```
UpdateBillboards()
```

```
GotoAndPlay("show")
```

```
MovieClip X Y
```

```
GFx::Value::SetDisplayInfo()
```

```
GFx::Value::SetText()
```

```
textField htmlText
```

```
HTML
```

```
>GetRank()+1
```

```
img
```

```
{1}
```

```
friendlies[i].pPlayer->GetName()
```

```
1
```

```
src='rank{0}'
```

```
friendlies[i].pPlayer-
```

```
vspace
```

5 Minimap ビュー

Minimap C++

Apps\Kits\HUD

- **FxMinimap.h**: minimap
- **FxMinimapView.h/.cpp**: minimap

5.1.1 Minimap ビュー

minimap

- **FxMinimapIconType**: ID
- **FxMinimapIcon**:
- **FxMinimapIconCache**:

MinimapIconFactoryBase

MovieClip

MovieClip

MovieClip

minimap

textField

Update()

o *PlayerIcon*:

o *FlagIcon*

o *ObjectiveIcon*

o *WaypointIcon*

- **FxMinimapView**: minimap

UpdateView()

PlayerIcon::Update()

Direct Access API

MovieClip

SetMember()

FxMinimapView.cpp 116

MovieClip

SetDisplayInfo

MovieClip

GFx::Value::SetMember

GFx::Movie::SetVariable

`virtual void Update(FxMinimapView* pview, FxMinimapEntity* pentity,`

```

float distSquared)

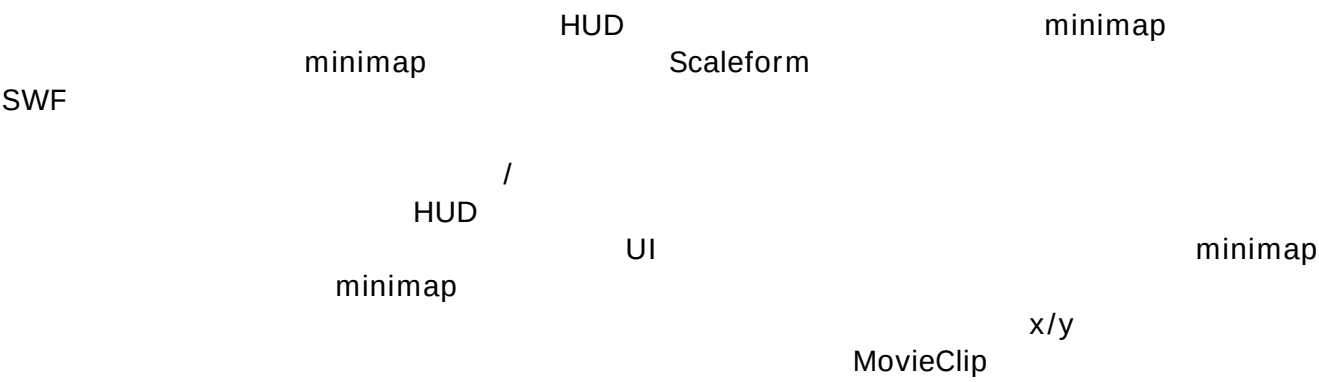
{
    SF_UNUSED(distSquared);
    bool hasIcon = (pentity->GetIconRef() != NULL);
    PointF pt = pentity->GetPhysicalPosition();
    bool showDir = pview->IsShowingPlayerDir();

    // If entity did not have an icon attached before, then wait for the
    // next update to set the state data. picon is most probably not
    // initialized (the movie needs to advance after creation)
    if (hasIcon && (State != (int)showDir))
    {
        // state 0: no arrow, state 1: show arrow
        Value frame(Double(showDir ? 2.0 : 1.0));
        MovieClip.Invoke("gotoAndStop", NULL, &frame, 1);
        State = (SInt)showDir;
    }

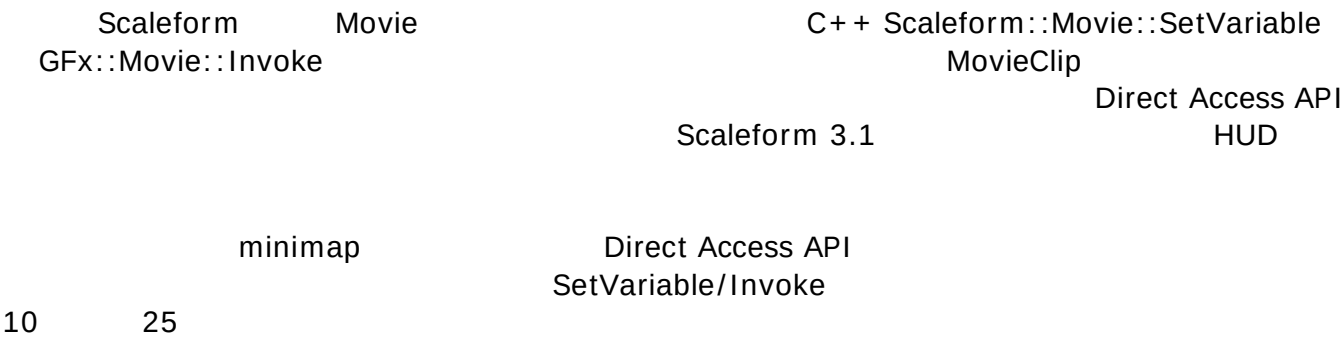
    pt = pview->GetIconTransform().Transform(pt);
    Value::DisplayInfo info;
    if (State)
    {
        info.SetRotation(pview->IsCompassLocked() ?
            (pentity->GetDirection() + 90) : (pentity->GetDirection() -
                pview->GetPlayerDirection()));
    }
    info.SetPosition(pt.x, pt.y);
    MovieClip.SetDisplayInfo(info);
}

```


6 性能解析



6.1 性能統計



6.2 メモリの分析

HUD

 GFXExport
 .GFX

1. ファイルフォーマット : GFX スタンダードエクスポート、イメージ非圧縮

全メモリ: 1858 KB

- HUDKit.gfx - 33 KB
- Minimap.gfx - 50 KB
- fonts_en.gfx - 46 KB
- gfxfontlib.gfx - 100 KB

非圧縮イメージ

- HUDKit.gfx
 - o UI - 603 KB
 - o - 72 KB
 - o - 51 KB
 - 備考: HUD 43 KB
- Minimap.gfx
 - o - 644 KB
- Map.jpg - 259 KB